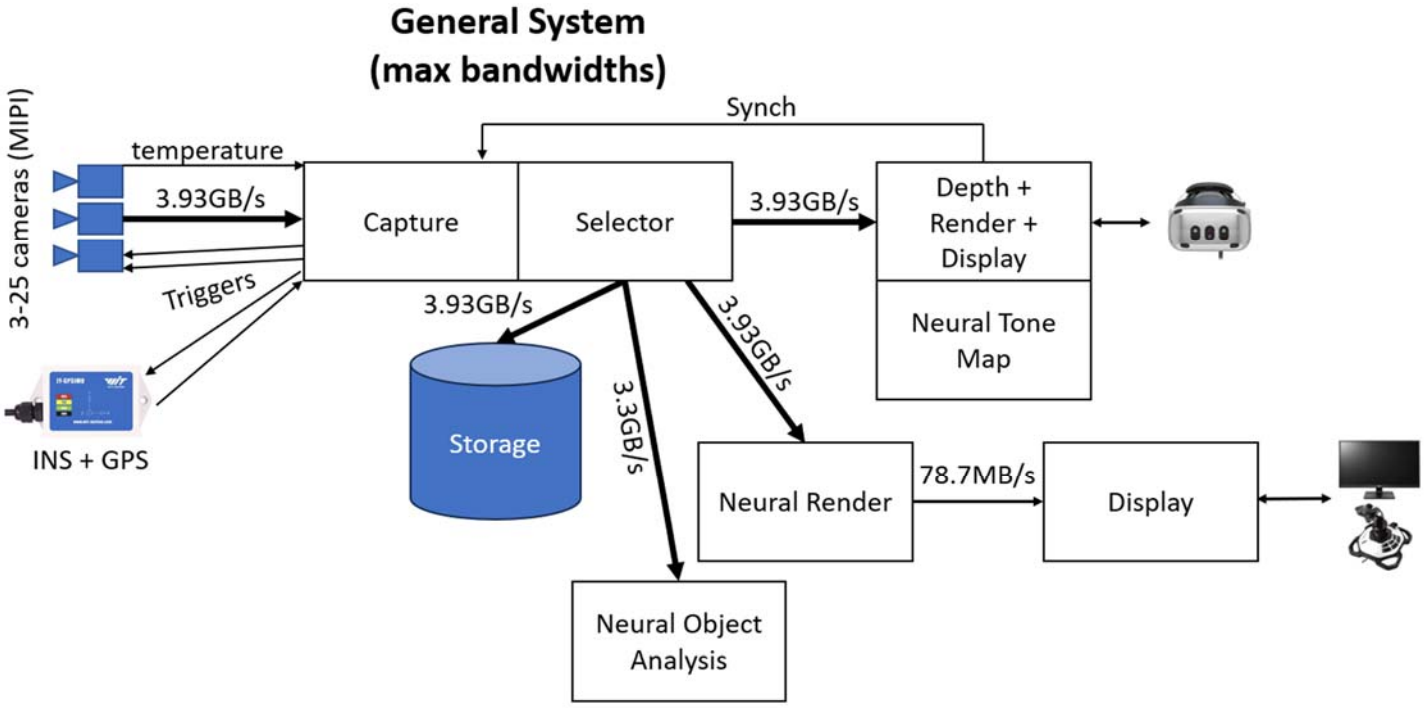
Software Architecture

This document describes the software architecture for the Apache IR strap-down pilotage project. It is based on the presentations in *TR001v04_System_Latencies*, *TR002v03_Camera_Layout*, *TR003v05_Camera_Distortion*, and *TR004v06_System_Architecture* documents along with the decisions made at the 11/13/2023 *Advanced Pilotage Sensor Characteristics* design review and hardware implementation design decisions through 9/7/2025. Document version 13 switched from UDP to TCP transport for most APIs, drastically changing the design from version 12. It also bumps the architecture version to 2.0.0. Changing the longitude, latitude, and altitude values from 32-bit to 64-bit floats bumped the major version to 4.0.0. Document version 24 replaces the begin/data/end frame video streaming messages with a common message to simplify FPGA implementation and accommodate restrictions on the FPGA UDP library being used, bumping the major version to 7.0.0. Document version 25 removes the extra frame-start time field from the common UDP video message and adjusts the meaning of the message time, bumping the major version to 8.0.0. Document version 30 adjusted the common UDP video message time once more and added two timing-related fields, bumping the version to 8.6.0; enabling these fields to be zero provides backwards compatibility.

System Architecture

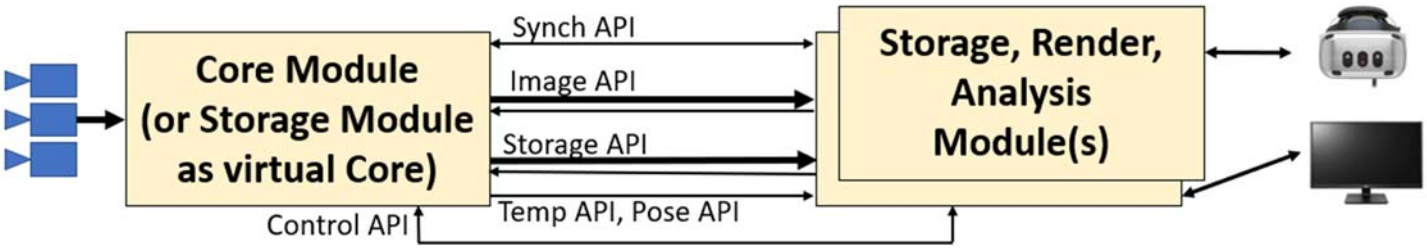


The diagram above, from *TR004v06_System_Architecture*, includes analytic and neural components. Not all system components will be used at the same time (only two 3.93GB streams are guaranteed to be supported). Maximum bandwidths are shown on the links. The system can operate with infrared cameras (IR) or with monochromatic visible-light cameras (VIS).

Design Decisions

- **Point-to-point 100 gigabit fiber Ethernet:** Provides a client-side interface that is sufficiently fast, broadly available, without special device drivers, perhaps through a switch. Adapters are available to convert this to Thunderbolt if needed.
- **TCP+UDP:** The main control algorithm will be run on a CPU on the FPGA board over TCP, but the FPGA control requires simplicity, precluding complex and general protocols. The dedicated point-to-point network avoids packet loss, so UDP will be used for image frame data coming from the FPGA fabric.
- **Separate camera streams:** Each camera will stream to endpoint(s) not shared with other cameras so that the FPGA streams can be separated from the CPU streams and from each other.
- **Implicit header size:** The header-size field can be determined based on the version of the protocol that is being used, so is not needed in the data stream and was removed moving from version 1 to 2.
- **"Magic cookie" and version only in TCP and Discovery:** The version number will be included at the start of the TCP stream and in the Discovery message, after the magic cookie to identify the stream type. These will not be present in other packets or messages.
- **Messages encapsulated in packets:** Each UDP camera stream requires a mechanism for the receiver to know if it has missed any packets. A sequence number is used to provide this. To make Message management the same between streams and to avoid having to unpack/repack during storage, these will also be used for Messages sent through TCP. This also provides a way to encapsulate multiple Messages (temperature messages, for example) in the same sent packet when using TCP. This will therefore be retained in version 2.
- **Unified video stream messages:** To simplify FPGA programming and to accommodate the FPGA UDP library being used, the three different stream messages (begin, data, end) were consolidated into a single message type on 5/16/2024.
- **Jumbo packets:** Jumbo Ethernet packets will be configured to 9000 bytes, enabling 8968-byte UDP payloads.
- **Only MIPI cameras:** The potential 3kx3k 3-camera system will not be included in the design.
- **Multi Core synch:** If two or more Core modules are needed to handle enough cameras, the clocks on the Cores must be synchronized with each other. One will provide a master clock that the others will use.

System Components



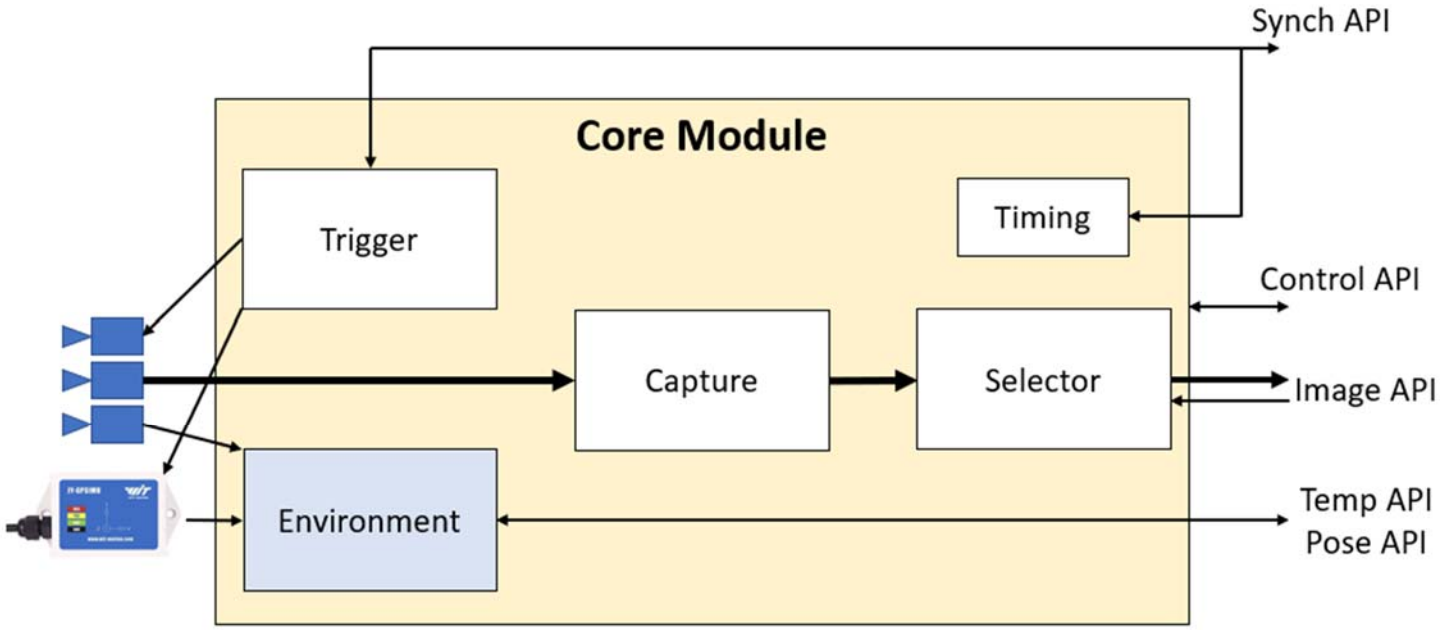
The system consists of the following modules, each of which communicates with the others via defined interfaces (APIs):

- **Core:** This module is attached to the cameras. It is responsible for triggering the cameras to produce synchronized frames and includes optional submodules for measuring environmental sensors (vehicle motion and camera temperatures) with time values synchronized with the video streams.
- **Storage:** This module provides a way to store data during a flight and then replay it later. It also provides a mechanism to store while providing live pass-through of data. It exposes a Storage API to control this.
- **Render:** This module communicates with a Core or Storage Module, is driven by one or more user input devices, and produces stitched images on one or more display devices. It includes several submodules, some of which are optional.
- **Analysis:** This module also communicates with a Core or Storage Module, but it produces analytical outputs (object detection events, for example) rather than display output.

Each of these modules is described below.

Calibration: There are out-of-band calibration procedures applied to the camera array. These always include geometrical calibration (relative positions, orientations, fields of view, and distortion correction) and may include brightness calibrations (vignetting, per-pixel gain and offset, etc.). These are described in a separate technical report. They produce a set of calibration files stored by camera serial number and date of calibration. These files can be read by the Rendering and Analysis modules and used to handle camera configuration and distortions.

Core Module



The *Core Module* is shown above, along with its submodules and interfaces. (Optional submodules are shown in blue.) Data flow is shown with arrows. (Thicker arrows indicate more data.)

When optional modules are not present, they have no effect and messages sent to them are ignored.

The submodules and APIs are described below.

See Appendix B for example client-side operations using the APIs.

Note: A same-computer virtual Core Module implementation could read from one or a series of files containing streams that have been copied from a Storage Module. It could provide a loopback-network same-computer replay by listening on port 10101 and routing messages from all incoming packets to any specified endpoints. It would also send Discovery messages (described below) to port 10102 on the loopback interface.

Network interface

The APIs described below TCP and UDP on a 100GigE NIC that has an unrouted class-C Martian address space that is specified at build time, perhaps 10.42.200.*. The NIC address in the address space is specified at build time (perhaps 10.42.200.1). The system does not run DHCP as either a client or a server. All multi-byte values are sent in little-endian order. Jumbo packets are used so that packets may be up to 8,968 bytes long. 4-byte fields in the packets are aligned to 4-byte boundaries, zero padded after the previous field when needed as described below.

Time fields are 4-byte unsigned seconds and 4-byte unsigned microseconds since the device was last reset (perhaps after last power-on) and are always specified in the Core Module time frame of reference.

The Core Module can be requested to send *messages* to an *endpoint* (IP address + port). When this happens, messages of all types requested for that endpoint are sent in the same *stream* and can be packed together into the same packet, interleaved by stream in any order but with ordering preserved within a stream.

Remote modules listen for Discovery packets sent from Core and Storage modules. These packets indicate the IP address and port to establish a TCP connection to, over which Commands flow to the Core and Messages flow to the remote module. Frame data is streamed over UDP from the Core to specified IP/port endpoints on the remote, with one endpoint per camera.

Remote submodules send command packets (which may initiate or modify message streams) by sending messages over a TCP connection to the IP/port specified in the Discovery packets (port 10101 by default). The first four bytes in the stream are the ***magic cookie "ASDP"***, then a 4-byte version, then a ***stream of commands***. Each Command has the following elements:

Field name	Length (bytes)	Description
Size	4	Total size of the packet in bytes (unsigned)
Operation code	4	Describes the operation to be performed
Parameters	*	Varies by the operation, padded to 4-byte boundaries

Command format

Operation codes are grouped by API: 0-9999 Control, 10000-19999 Synch, 20000-29999 Image, 30000-39999 Storage, 40000-49999 Temp, 50000-59999 Pose.

Core submodules send ***all messages besides Discovery and image frame information on the same TCP connection*** that the remote module uses for Commands. Again, the first four bytes in the stream are the ***magic cookie "ASDP"***, then a 4-byte version, then a stream of messages. ***Discovery and image frame data is sent packed inside UDP packets.*** Each UDP packet has the following header information:

Field name	Length (bytes)	Description
Size	4	Total size of the packet in bytes (unsigned)
Sequence number	4	Starts at 0, increments for all packets to this endpoint
Messages	*	Zero or more messages, padded to 4-byte boundaries

Core Module UDP data stream packet format

The Core Module message format is as follows:

Field name	Length (bytes)	Description
Size	4	Total size of the message in bytes (unsigned)
Time	8	The time of the message (time of first data value in message)
Message type	4	Indicates message type and implicitly the API
Parameters	*	Varies by message type, padded to 4-byte boundaries

Core Module message format

Trigger Submodule (see Synch API)

Provides frequency- and phase-controlled hardware and software output triggers needed by sets of cameras and environmental sensors. It can receive synchronous or one-shot hardware and software input triggers from the *Synch* API and it aligns its output triggers with these in frequency and phase. It controls at least four hardware triggers that can be connected to subsets of the cameras and/or subsets of the environmental sensors. It provides at least 255 external software and at least four external hardware triggers.

Timing Submodule (see Sync API)

Provides a microsecond-accurate timing reference for all other submodules of the Core module. This time reference is monotonically increasing and steady, without corrective jumps or changes in rate.

This module also provides an interface to log time-stamped event messages that all submodules can use to time-stamp events (start frame receipt, end frame receipt, start frame transmit, …) along with string messages telling parameters (which camera) and other information. This is used for clock synchronization on connected Modules. It can also be used for performance tuning (for example, determining the optimal relative offsets for triggers for specific system configurations) and for debugging.

Capture Submodule (see Control API, Storage API)

Provides routing of incoming pixels from each camera in the system, timestamping and sending each scan line (or group of scan lines) as it arrives from the sensor without buffering the entire image before sending. Its routing is configured at system start and can include up to two target submodules. It can be configured at run time to read from either the camera hardware or the Storage module. It also tracks per-frame metadata (exposure and gain values from the VIS cameras).

Environment Submodule (see Temp API, Pose API, Trigger API)

This optional submodule provides configuration control over and time-stamped measurements of readings from the temperature sensor attached to each camera (if they are present) and the GPS/INS navigation system (if it is present). These are sent via the *Temp* (temperature) API and *Pose* API to all connected modules.

Selector Submodule (see Image API)

Provides streaming of image subsets to connected modules via the *Image* API. An attached module can request a set of streams. Each stream is from a single camera, and it specifies how many frames to skip (0 or more), the region of the image to send (up to the whole image). It also streams per-frame metadata (gain and exposure for VIS cameras) along with the image data.

Temp (temperature) API

This optional API provides a mechanism for a client module to stream temperatures from all system sensors.

It uses the following network protocol:

- Operation codes handled:
  - 40000: Stream temperatures (no parameters). Begins streaming temperature measurements as they arrive. By default, temperature data is not streamed.
  - 40001: Cancel temperature streaming (no parameters). Cancels streaming.
- It sends the following message codes:
  - 40000: Temperature. A report for a single sensor, whose time is in the message time field. The system sends messages for all sensors as new data arrives from them. It has the following parameters:
    - 2-byte unsigned camera ID (0 for system, 1-N for cameras) that the measurement is on.
    - 2-byte unsigned sensor ID, indicating which temperature sensor on that camera/system (first ID is 1).
    - 4-byte IEEE 32-bit float reporting the temperature measurement in Celsius.

Pose API

This optional API provides a mechanism for a client module to query the pose (position, orientation, velocity, and rotational velocity) at a given time. This is provided by the Core module streaming time-stamped measurements as they arrive and by client-side query functions that receive the stream and provide the estimated values from of a given sensor (GPS, INS) at a given time.

It uses the following network protocol:

- Operation codes handled:
  - 50000: Stream poses (no parameters). Begins streaming measurements as they arrive.
  - 50001: Cancel pose streaming (no parameters). Cancels streaming.
- It sends the following message codes:
  - 50000: Pose. A pose and velocity estimate, whose time is in the message time field. The time is the actual device reporting time for the most-recent pose, which may be repeated if the report rate is faster than the device sampling rate. For Kalman or other optimal-estimation approaches, it is the time that the

estimate was made. The rotation fields are in degrees around the X, Y, and Z axes in that order. The canonical orientation has X pointing East, Y pointing North, and Z pointing up (undefined at the poles). The helicopter orientation is defined as pilot right is X, pilot forward is Y, and up is Z. The rotation tells how to re-orient an object with canonical orientation to the helicopter orientation. The linear velocity is expressed in local rotated helicopter coordinate: a +Y value means that the helicopter is flying forwards. The rotation velocity estimates are also in local rotated helicopter coordinates: a +X value means that the helicopter is tilting back. The Pose has the following parameters:

- 8-byte IEEE 64-bit float reporting the estimated longitude in degrees (-1e6 for none estimated).
- 8-byte IEEE 64-bit float reporting the estimated latitude in degrees (-1e6 for none estimated).
- 8-byte IEEE 64-bit float reporting the estimated altitude above sea level in meters (-1e6 for none estimated).
- 4-byte IEEE 32-bit float reporting the estimated rotation about X in degrees.
- 4-byte IEEE 32-bit float reporting the estimated rotation about Y in degrees.
- 4-byte IEEE 32-bit float reporting the estimated rotation about Z in degrees.
- 4-byte IEEE 32-bit float reporting the estimated linear velocity in X in meters/second (0 if no position is estimated).
- 4-byte IEEE 32-bit float reporting the estimated linear velocity in Y in meters/second (0 if no position is estimated).
- 4-byte IEEE 32-bit float reporting the estimated linear velocity in Z in meters/second (0 if no position is estimated).
- 4-byte IEEE 32-bit float reporting the estimated rotation velocity around X in degrees/second.
- 4-byte IEEE 32-bit float reporting the estimated rotation velocity around Y in degrees/second.
- 4-byte IEEE 32-bit float reporting the estimated rotation velocity around Z in degrees/second.

Image API

Provides a run-time-controllable set of streams from the image streams from each camera. Each connected module can specify one or more sets of stream parameters, and each receives its own set of messages. The Core module streams time-stamped image subsets to the client side (bounded by start/stop messages so the client can know when the entire frame has been received for a given camera). The client can use this to provide per-camera streaming interfaces to support progressive calculations and transfer to GPU memory.

It uses the following network protocol:

- Operation codes handled:
  - 20000: Stream subregion. Begins streaming subregions to a specified UDP endpoint as relevant images arrive. Updates parameters if already streaming for this camera (each camera can stream to only a single endpoint per remote Module). Changes to the parameters for the specified cameras on the Selector submodule are made after any current frame has completed sending for a given camera. When the TCP connection to a client shuts down, all ongoing camera streams also shut down. Parameters follow:
    - Camera ID: 4 bytes; 1-25.
    - IP: 4 bytes; IP address of the host to stream to.
    - Port: 4 bytes; port to stream to (largest value is 2 bytes, padded on the right with 0).
    - Skip frames: 4 bytes; number of frames to skip (0 for sending every frame, 1 for every other frame).
    - Start time seconds: 4 bytes; Time to start streaming from the camera. Wait for the first frame whose first pixels arrive at or after this time to begin streaming. If this time is in the past, stream the next available frame. 0 for “immediately”.

        - Start time microseconds: 4 bytes; Time to start streaming from the camera. Wait for the first frame whose first pixels arrive at or after this time to begin streaming. If this time is in the past, stream the next available frame. 0 for “immediately”.
        - Left: 2 bytes; Left side of the region in pixels; 0 for left side of the image.
        - Top: 2 bytes; Top of the region in pixels; 0 for top of the image
        - Right: 2 bytes; Right side of the region in pixels; NX-1 for the whole image.
        - Bottom: 2 bytes; Bottom side of the region in pixels; NY-1 for the whole image.
    - 20001: Cancel subregion streaming. Cancels streaming for the specified camera. Has no effect if there is not an ongoing stream for the specified camera and endpoint. Parameters follow:
        - Camera ID: 4 bytes; 1-25.
        - IP: 4 bytes; IP address of the host to stream to.
        - Port: 4 bytes; port to stream to (largest value is 2 bytes, padded on the right with 0).
- It sends the following message codes (note removal of three types in version 5.0.0):
    - ~~20000: Begin frame. Reports the beginning of the frame along with frame metadata. The time of the message is the time at which the first pixel from the image arrived, even when it is not in the subregion. It precedes all frame data messages for a frame. It has the following parameters:~~
        - ~~Camera ID: 4 bytes signed; 1-25.~~
        - ~~Camera Type: 4 bytes; Table indicating the camera type: lens/sensor on this camera. This also indicates whether the camera is an IR or VIS camera and its brand and model. This provides information that the client needs to know to determine which metadata fields are filled in.~~
        - ~~Sensor size in X: 2 bytes; Number of pixels in X.~~
        - ~~Sensor size in Y: 2 bytes; Number of pixels in Y.~~
        - ~~Exposure: 4-byte IEEE 32-bit float reporting the exposure time in seconds (0 for none reported).~~
        - ~~Gain: 4-byte IEEE 32-bit float reporting the sensor gain multiplier (0 for none reported).~~
        - ~~Reserved: 128 bytes reserved for future metadata fields.~~
    - ~~20001: Frame data. Contains data describing a rectangular block of pixels in the frame in a camera. Many of these may be reported to completely describe the subregion (or set of subregions), due to packet size limitations. Messages are sent as new lines arrive from the camera (not waiting for a complete frame). It has the following parameters:~~
        - ~~Camera ID: 4 bytes signed; 1-25.~~
        - ~~Left: 2 bytes; Left side of the region; 0 for left side of the image.~~
        - ~~Top: 2 bytes; Top of the region; 0 for top of the image~~
        - ~~Right: 2 bytes; Right side of the region; NX-1 for the right side of the image.~~
        - ~~Bottom: 2 bytes; Bottom side of the region; NY-1 for the right side of the image.~~
        - ~~Values: 2 bytes per pixel, starting at the upper-left pixel in the block and proceeding to the right, followed by the next line. Padded with 2 bytes at the end if there are an odd number of pixels.~~
    - ~~20002: End frame. It comes after all frame data messages for a given frame in a camera. Its time matches the time of arrival of the last pixel in the image, even when it is not in the subregion. Parameters follow:~~
        - ~~Camera ID: 4 bytes signed; 1-25.~~
    - 20003: Consolidated frame data. Contains data describing a rectangular block of pixels in the frame in a camera (it may be an empty block). It includes flags indicating the beginning and end of a frame along with additional timing information for the beginning of the frame and its duration. ~~The time of the message itself depends on the flags: if neither the begin or end flag is set, the time is that of the last pixel in the message data block; if only the begin frame flag is set, the time is that of the first pixel in the image even if it is not in the subregion being scanned; if only the end frame flag is set, it is the time of the last pixel arrival from the camera even if that pixel is outside of the subregion being scanned; If both flags are set, it is the average of the times for the first and last pixels in the frame even if they are not in~~

~~the subregion being scanned.~~ The time of the message itself is the time of the first pixel stored in the message. It has the following parameters:

- Camera ID: 32-bit little-endian; 1-25.
- Camera Type: 4 bytes; Table indicating the camera type: lens/sensor on this camera. This also indicates whether the camera is an IR or VIS camera and its brand and model. This provides information that the client needs to know to determine which metadata fields are filled in.
- Sensor size in X: 2 bytes; Number of pixels in X.
- Sensor size in Y: 2 bytes; Number of pixels in Y.
- Left: 2 bytes little endian index of the leftmost pixel in the data region in this message (0 for left side of image, 1 for an empty message).
- Top: 2 bytes little endian index of the topmost pixel in the data region in this message (0 for top of image, 1 for an empty message).
- Right: 2 bytes little endian index of the rightmost pixel in the data region in this message (NX-1 for right side of image, 0 for an empty message).
- Bottom: 2 bytes little endian index of the bottom pixel in the data region in this message (NY-1 for bottom of image, 0 for an empty message).
- Flags: 32-bit little-endian, with LSB indicating the beginning of a frame (subregion) and the second bit indicating the end of a frame (subregion); both can be set for a very small subregion. The next two bits indicate whether the frame number will be packed and whether the line counter will be packed. Other bits are reserved and set to 0.
- Exposure: 4-byte IEEE 32-bit float reporting the exposure time in seconds (0 for none reported).
- Gain: 4-byte IEEE 32-bit float reporting the sensor gain multiplier (0 for none reported).
- Frame number: 16-bit little-endian that starts at 0 for the first frame sent. It may reset to zero during the run. It increments by one every time a frame is sent, even when the stream is set to skip frames. This should be zero if the third flag bit is zero.
- Line counter: 16-but little-endian line count. This is based on the first line in Values if there are multiple lines and it is numbered starting with 0 from the top of the whole sensor, so will not start with 0 if the top of the image is not in the subregion being sent on this stream. This should be zero if the fourth flag bit is zero.
- Frame begin time: 32-bit little-endian seconds of the time of the first pixel in the frame (even if it is not in the subregion) followed by 32-bit little-endian microseconds (8 bytes total). This value may be zero, in which case the meaning of the message time is described by the struck-through text above.
- Frame duration: 32-bit little-endian microseconds recording the duration from the first pixel to the last pixel in the previous video frame (even if those pixels are not in the region). This value may be zero, in which case the meaning of the message time is described by the struck-through text above.
- Reserved: 112 reserved bytes that must be all zeroes.
- Values: 2 little-endian bytes per pixel, starting at the upper-left pixel in the message and proceeding to the right, followed by the next line. Each line is padded on the right with 2 bytes of arbitrary value if there are an odd number of pixels to make each line always a multiple of 4 bytes in total length.

Synch API

Provides translation between the *Timing* submodule's local clock and each connected module's local clock. Provides control over hardware triggering of the *Trigger* submodule. Provides one-shot and periodic software triggering of the *Trigger* submodule from the client side based on the client's local clock at the time of the trigger, and adjustment of the timing of future triggers based on drift in a software trigger.

It uses the following network protocol:

- Operation codes handled:
  - 10000: Configure trigger. Enables a client to configure the trigger state of one of the available hardware triggers. **Note:** At system boot, all triggers are disabled. Parameters follow:
    - Period: 8 byte IEEE double-precision floating-point value with period in seconds.
    - Offset: 4 byte IEEE floating-point value with offset from incoming trigger in seconds. A positive offset will cause the local hardware trigger to fire later than the incoming trigger command. Negative values are clamped to 0.
    - Tracking factor: 4 byte IEEE floating-point value in [0-1] telling fraction of discrepancy to correct when synchronizing with a new incoming trigger. A value of 1 completely synchronizes with each trigger. A value of 0.1 shifts by $1/10^{th}$ of the distance, smoothing the adjustment to handle jitter in the incoming hardware trigger while following long-time-scale drift.
    - ID: 2 bytes; unsigned ID of the trigger to configure, starting with 1.
    - Mode: 1 byte unsigned; bit 0 = periodic, bit 1 = responds to software trigger, bit 2 = responds to hardware trigger. If bit 2 is set, bit 1 is ignored (hardware triggering dominates software triggering). (type: 0 = disabled, 1 = unsynchronized, 2 = one-shot software, 3 = periodic software, 4 = one-shot hardware, 5 = periodic hardware).
    - External ID: 1 byte; Index of the incoming software or hardware trigger (if any) to fire based on. The first trigger ID is 1. Ignored for disabled and unsynchronized modes.
  - 10001: Software trigger. Parameters follow:
    - ID: 1 byte; ID of the software trigger being sent. Padded on the right by 3 0 bytes. When a one-shot trigger is received, it causes any associated output triggers to fire at the specified phase from the specified time. When a periodic trigger is received, the time is first adjusted by the exponential filter before the offset is applied; this enables the remote trigger to jitter while keeping the output trigger both periodic and aligned with the connected module. The nearest-in-time scheduled trigger is used to adjust expected trigger time.
    - Time Seconds: 4 bytes; time in camera clock to fire the trigger.
    - Time Microseconds: 4 bytes; time in camera clock to fire the trigger.
  - 10002: Set event verbosity. Controls which events are sent to the remote. Default is all events. Parameters follow:
    - Priority verbosity: 1 byte; Specifies the maximum priority value of a message to be delivered. Messages with higher values are not sent. Defaults to 0. System control events cannot be filtered. Larger numbers can be set to receive additional information that can be useful for system debugging and throughput/latency measurement.
    - Reserved: 3 bytes; set to 0.
- It sends the following message codes:
  - 10000: Event. Description of an event that has been reported by any Core Submodule. It has the following parameters:
    - Priority: 1 byte; 0 = system control, 1 = very high, higher are at decreasing levels of importance. Priority 0 and 1 event messages are sent out-of-band, in their own packets.

- Type: 3 bytes; Specifies the message type. This can enable fast parsing of messages without reviewing their string contents. Types 0-255 are reserved for critical errors, types 256-511 are reserved for errors, types 512-767 are reserved for warnings, and higher numbers are informational. See Appendix A.
- Start: 4 bytes; offset to the character-string portion of the message (size of the headers in the parameter set: this value is 8 for version 2.0.0).
- String: N bytes, padded with zeroes to reach a multiple of 4. UTF-8 encoded string describing the message, perhaps with embedded parameters that must be parsed from the string. Intended for presentation to the end user. See Appendix A.

The client module can use event type 768 (clock sync) to estimate the offset and rate difference between its local clock and the Core Module clock and use that to convert trigger times.

Control API

Provides the ability to determine available submodules and other APIs. Provides the ability to configure power-on behavior to enable automatic recording during flights without a connected module. Provides the ability configure various system states (record, playback, live).

It uses the following network protocol:

- Operation codes handled:
  - 0: Reset. No parameters. Resets the system as if it were powered off and back on. This command should not be commonly used because it will destroy streams to all clients and restart storage. Useful to set the system into a known state after a system failure.
  - 2: Start recording (sent to server side of Storage module). No parameters. Selects the next-available integer ID by increasing the largest existing ID by one. If the system was already recording, has no effect. Has no effect if sent to a Core module that is not also a Storage module.
  - 3: Stop recording (sent to server side of Storage module). No parameters. Closes any open file and stops recording. If the system is not recording, has no effect. Has no effect if sent to a Core module that is not also a Storage module.
  - 4: Start replay (sent to server side of Storage module). If the system is recording, has no effect. Has no effect if sent to a Core module that is not also a Storage module. If the system has any open streams on any NIC, they are terminated. Inputs from the Capture and Environment Submodules are ignored, as are Events from all Core Submodules. Packets are read from disk and sent through the system at the rate indicated by their relative time stamps, starting at the specified time; the last packet may be partial and if so it is not transmitted. These are sent to any new streams as requested. When the end of the stored session is reached, no more packets are delivered. Status messages are updated based on system state.
    - ID: 4 bytes; ID of the saved stream to replay.
    - Time Seconds: 4 bytes; time in camera clock to commence replay.
    - Time Microseconds: 4 bytes; time in camera clock to commence replay.
  - 5: Pause replay (sent to server side of Storage module). No parameters. Pauses time during replay, causing no new messages to be sent. If the system is not replaying, has no effect. Has no effect if sent to a Core module that is not also a Storage module.
  - 6: Resume replay (sent to server side of Storage module). No parameters. Resumes time during replay from the time it was paused, causing new messages to be sent starting with the previous time. If the system is not paused during replay, has no effect. Has no effect if sent to a Core module that is not also a Storage module.

- 7: Stop replay (sent to server side of Storage module). No parameters. Closes any open file and stops replay. If the system is not replaying, has no effect. Has no effect if sent to a Core module that is not also a Storage module. This switches a Storage module back into Live mode if it is connected to a Core module.
- 8: Set start-up recording state (sent to server side of Storage module). Determines whether the system will start recording after a reset or power cycle. Has no effect if sent to a Core module that is not also a Storage module. It has the following parameters:
  - State: 4 bytes; Value of 0 for not saving at startup, 1 for saving at startup.
- 10: Set stream state period. Sets the period with which state messages are sent. Parameters follow:
  - Period: 4 bytes; 32-bit IEEE floating-point value in seconds.
- 11: Set NUC flag state. This sets the state of the physical Non-Uniformity Correction flag that can cover the lens of a camera. The system will respond by sending a NUC_FLAG_STATE event message when the command is executed.
  - ID: 4 bytes; ID of the camera whose flag should be set.
  - State: 4 bytes; Value of 0 for hidden, 1 for covering camera.
- 12: Initiate parameterless on-camera NUC. This starts an on-camera Non-Uniformity Correction on the specified camera. The system will respond by sending an ON_CAMERA_NUC_STATE event when the NUC starts and another when it completes.
  - ID: 4 bytes; ID of the camera that should perform the NUC.

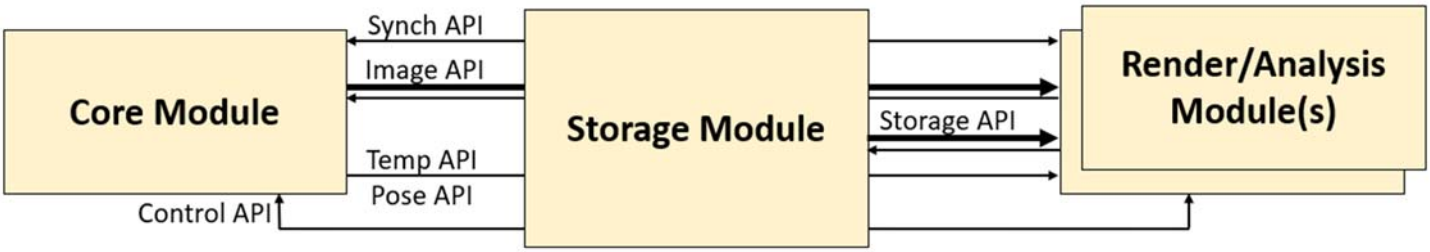
- It sends the following message codes:
  - 0: Discovery. This message is sent twice per second on each active NIC to port 10102 on the broadcast address for that NIC. It is encoded as the only message in a packet whose sequence number is always 0. It has the following parameters:
    - Magic Cookie: 4 bytes: “ASDP”.
    - Version: 4-byte version number: major (1 byte), minor (2 bytes), patch (1 byte).
    - IP: 4 bytes; IP address to connect TCP to.
    - Port: 4 bytes; port to connect TCP to (largest value is 2 bytes, padded on the right with 0).
    - Serial #: 4 bytes; Serial number of the system. Can be used to determine capabilities via table look-up.
  - 1: State. It has the following parameters:
    - Number of features: 4 bytes; count of the number of feature values following.
    - Features: List of 2-byte feature codes, padded to 4 bytes at the end; Values indicating the availability of optional features (submodules, etc.). 1 = Storage Module available, 3 = Temperature API available, 4 = Pose API orientation available, 5 = Pose API position available, 6 = NUC flag available, 7 = on-camera NUC available; other values reserved.
    - Number of cameras: 4 bytes; count of camera records following.
    - Cameras: List of records, one per count, in order of increasing index:
      - Min trigger period: 8-byte IEEE 32-bit double reporting the minimum period in seconds.
      - Max trigger period: 8-byte IEEE 32-bit double reporting the maximum period in seconds.
      - Trigger: 4 byte unsigned; Which internal hardware trigger tied to (0 for none).
      - Camera Type: 4 bytes; Table indicating the camera type: lens/sensor on this camera. This also indicates whether the camera is an IR or VIS camera and its brand and model. This provides information that the client needs to know to determine which metadata fields are filled in and how many temperature sensors are installed in what locations.
      - Sensor size in X: 2 bytes; Number of pixels in X.
      - Sensor size in Y: 2 bytes; Number of pixels in Y.

- Number of temperature sensors per camera: 4 bytes; How many sensors per camera. This is the maximum number of sensors across all cameras in the system; some cameras may have fewer. The actual number of sensors for a given camera should be determined based on the *Cameras/Camera Type* field.
- Number of system temperature sensors: 4 bytes; How many sensors not on cameras.
- Storing (set by Storage module): 1 byte; 1 if data being stored to disk, 0 if not.
- Cameras streaming (different for Storage and Core modules, indicates live mode on Storage module): 1 byte; 1 if camera data is streaming, 0 if not.
- Replaying (set by Storage module): 1 byte; 1 if data being replayed from disk, 0 if not.
- Replay at end (set by Storage module): 1 byte; 1 if at end of replay (no more data coming), 0 if not (or not replaying).
- Record on reset (set by Storage module): 1 bytes; 1 if will start recording after reset, 0 if not.
- Padding: 3 bytes; reserved for future use, set to 0.
- Number of hardware triggers: 4 bytes; How many hardware triggers are available.
- Trigger configuration: One record per hardware trigger, in order of increasing ID:
  - Period: 8 byte double-precision IEEE floating-point value with period in seconds.
  - Offset: 8 byte double-precision IEEE floating-point value with offset from incoming trigger in seconds.
  - Tracking factor: 4 byte IEEE floating-point value with fraction of offset to include when synchronizing with a new incoming trigger. A value of 1 completely synchronizes with each trigger. A value of 0.1 shifts by $1/10^{th}$ of the distance, smoothing the adjustment to handle jitter in the incoming hardware trigger while following long-time-scale drift.
  - ID: 2 byte unsigned.
  - Mode: 1 byte unsigned; bit 0 = periodic, bit 1 = responds to software trigger, bit 2 = responds to hardware trigger. If bit 2 is set, bit 1 is ignored (hardware triggering dominates software triggering). (type: 0 = disabled, 1 = unsynchronized, 2 = one-shot software, 3 = periodic software, 4 = one-shot hardware, 5 = periodic hardware).
  - External ID: 1 byte; Index of the incoming software or hardware trigger (if any) to fire based on. The first trigger ID is 1. Ignored for disabled and unsynchronized modes.
- Total disk space: 8 bytes; Size of the total disk storage in bytes.
- Remaining disk space: 8 bytes; Size of the remaining disk storage in bytes.
- Stream replay time seconds: 4 bytes; Time since start of stream (0 if not replaying).
- Stream replay time microseconds: 4 bytes; Time since start of stream (0 if not replaying).

o 10000: Events. This API can send the following events surrounding replay:

- 769: Start of replay is sent when replay begins in response to command 4.
- 770: End of replay is sent when replay stops in response to command 7.
- 773: Pause replay is sent when replay is paused in response to command 5.
- 774: Resume replay is sent when replay is resumed in response to command 6.

Storage Module



The *Storage Module* provides the ability to synchronously store data for sessions captured during a flight and then to replay it later. To do this, it implements both the client side of each API (to determine the available streams and request them all from a Core Module) and the server side (to stream information live or from disk). It also implements a new *Storage* API that enables listing, starting, stopping, and deleting stored streams. This module records data in *Live* and *Store* modes and produces data in *Live* and *Replay* modes. The three modes behave as follows:

- **Store:** In this mode, it implements the client side of each API and reads all available data from a Core module and stores it to a session on disk.
- **Replay:** In this mode, it implements the server side of each API; data is read from disk and selected subregions are sent to one or more modules including time data based on a clock that is synchronized with the other modules.
- **Live:** This mode performs a subset of *Store* and *Replay* together with forwarding of software triggers between an attached module and a Core module, providing pass-through data from a Core module.

Copying stored data files from a Storage module is done using the *SFTP* protocol[1], requiring a user account or public/private key pair to access. All data for a captured session is stored in a directory named by the session ID. The file formats and layouts within the session are implementation dependent.

When available storage fills up, storage stops and the final packet for each stream being stored in the session may have been truncated.

The system can be configured to start storing at power-up using the *Set start-up recording state* Control API command.

Control GUI: The module will run a web server on port 80 of both a GigE and WiFi connection that provides information about storage state, disk capacity, and so forth. It will enable control over whether the system is storing.

Passthrough GUI: The module will provide the ability to use SSH port forwarding to establish a connection to port 80 on a Core Module connected via GigE at a specified address. This is in addition to the 100Gbit Ethernet connection and is used to support diagnostics and image previewing on the device.

Storage API

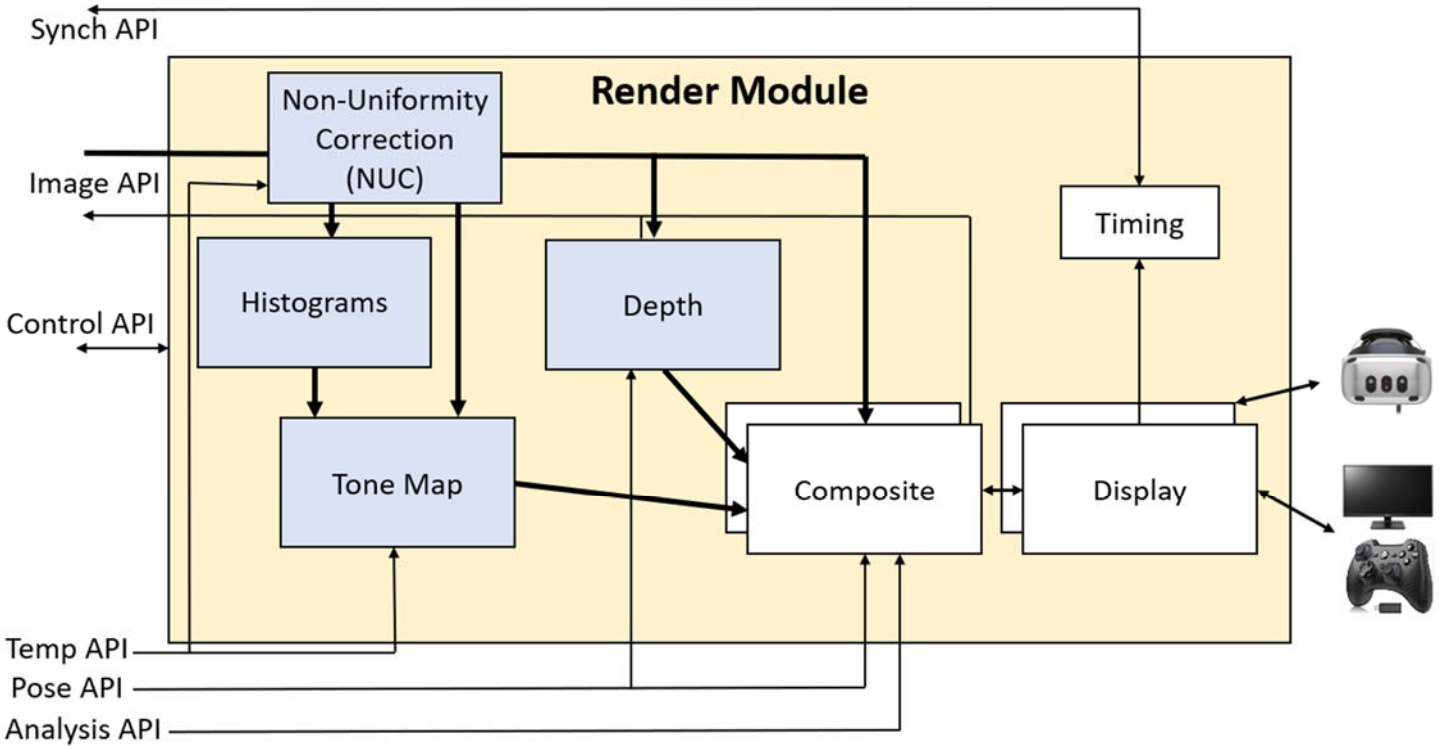
This additional API provides a mechanism to locate and delete the files stored on disk associated with stored sessions. It is exposed by a Storage Module in addition to the other APIs that it presents as a Core module (it can be thought of as a derived class from the Core Module). The stream list is used to determine the values to send to the *Start replay* Control API command.

It uses the following network protocol:

- Operation codes handled:
  - 30000: Erase all stored sessions. No parameters. Does not erase a stream that is currently open for writing.
  - 30001: List stored streams (no parameters). Sends a list of available stored stream IDs.
  - 30002: Erase stored stream. Does not erase a stream that is open for writing. Parameters follow:
    - ID: 4 bytes; ID of the saved stream to erase.
- It sends the following message codes:
  - 30000: Stream list. A complete list of available stored stream IDs. It has the following parameters:
    - Count: 4 bytes. Number of stored streams.
    - List of 4-byte IDs, as many as listed in Count.

[1] https://en.wikipedia.org/wiki/SSH_File_Transfer_Protocol

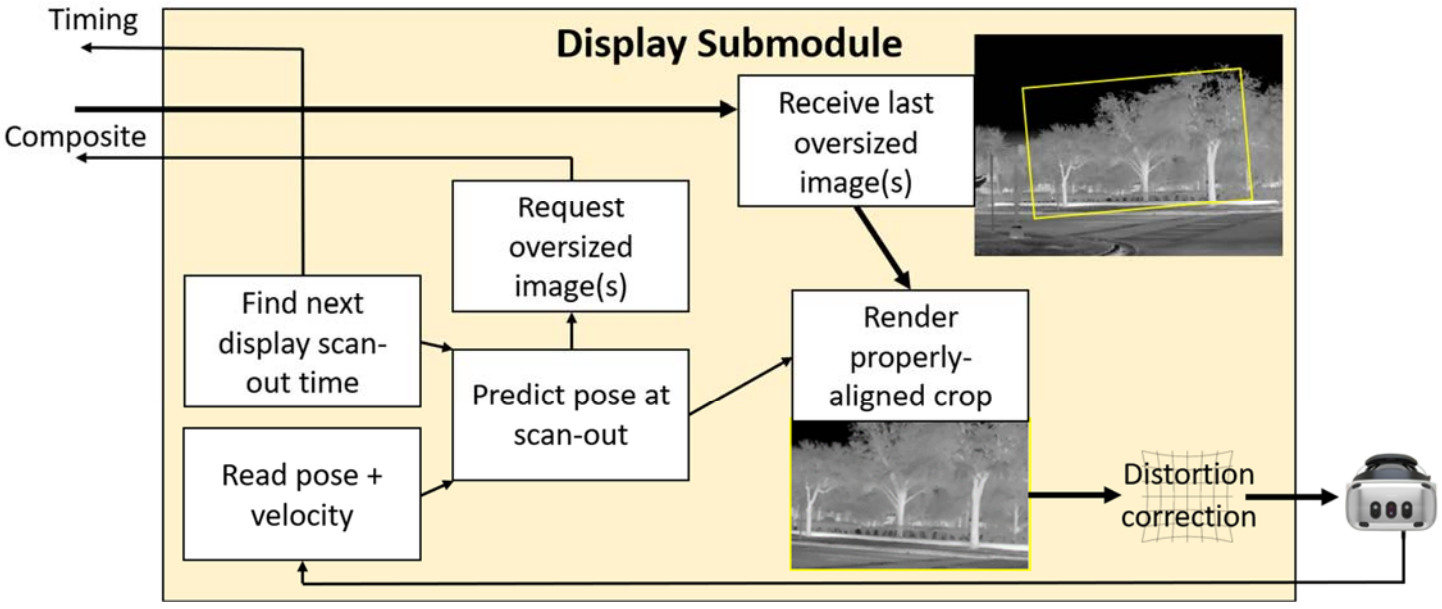
Render Module



The *Render Module* is shown above, along with its submodules and interfaces. (Optional submodules are shown in blue.) Data flow is shown with arrows. (Thicker arrows indicate more data.) The module is shown with two Composite + Display submodules for driving two displays from the same Render module.

The APIs used by this module are described in the Core and Analysis modules. Its submodules are described below.

Display Submodule



This submodule asks the Composite submodule for images at specific times in the local clock from virtual cameras whose poses are described in the pilot's coordinate system. This coordinate system has its origin at the resting location of the center of the pilot's head when looking straight forward and level. Translation and rotation of the head and any eye

offsets are described for each requested image. The image resolution and field of view may be oversized to support prediction error and distortion correction. One or two eyes may be asked for, with an optional pair of high-resolution inserts when using two eyes to support foveated rendering.

The Display submodule then warps the returned images based on updated head tracking data and HMD distortion and displays them. It uses the most recently available frame for each update, perhaps re-warping an old image if the Composite submodule does not respond quickly enough. A constructor parameter tells how long before frame scan to ask for a new frame (which should consider maximum display composite times).

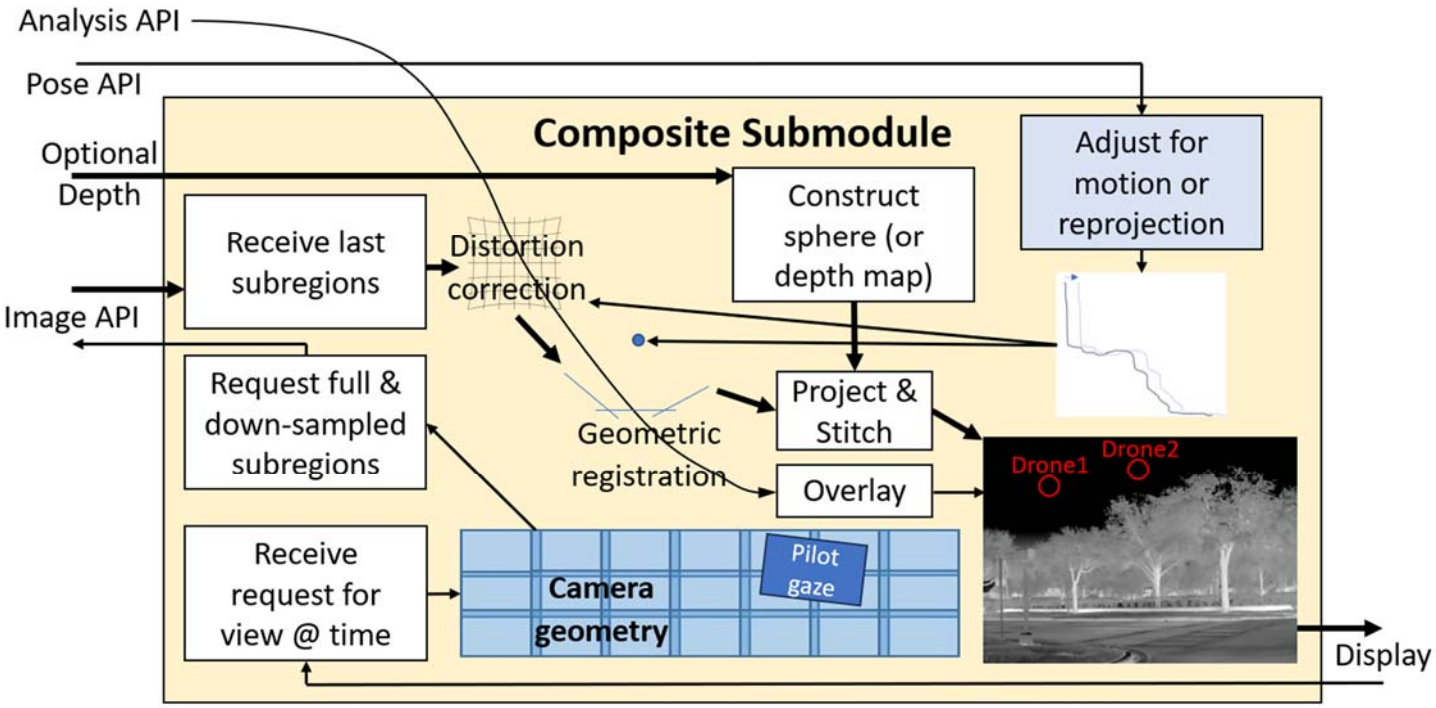
The Display submodule can also send the frame scan out start time as a software trigger to the Timing submodule to keep frame capture aligned with display scan out.

Timing Submodule

This is the client-side portion of the submodule described in the Core module.

Composite Submodule

The diagram below shows an analytical composite submodule; a neural submodule will behave differently.



This submodule receives requests from the Display submodule for one or more virtual-camera images from specified viewpoints at a specified time. For each viewpoint, it determines the subregions of the physical cameras that are overlapped by the region. It then modifies the parameters on the Image API to request the appropriate sub-regions from the subset of the cameras that overlap the viewed region. (It dilates the region to be able to hide prediction error and distortion correction warping.)

Meanwhile, the sub-regions being sent from the previous parameters are arriving. They have distortion correction applied to come from camera space into affine space and assigned relative positions and orientations according to the geometric camera calibration (these describe how the pixels from each camera will be projected into space from its individual center of projection).

A projection manifold is constructed describing the geometry that the pixels will be projected onto, which by default is a sphere with a specified radius. If depth information is available, the projection manifold is that geometry, which will have

each location at the estimated depth away from the center of projection of the entire camera array, enabling proper stitching of edges and different motion for near objects than far objects.

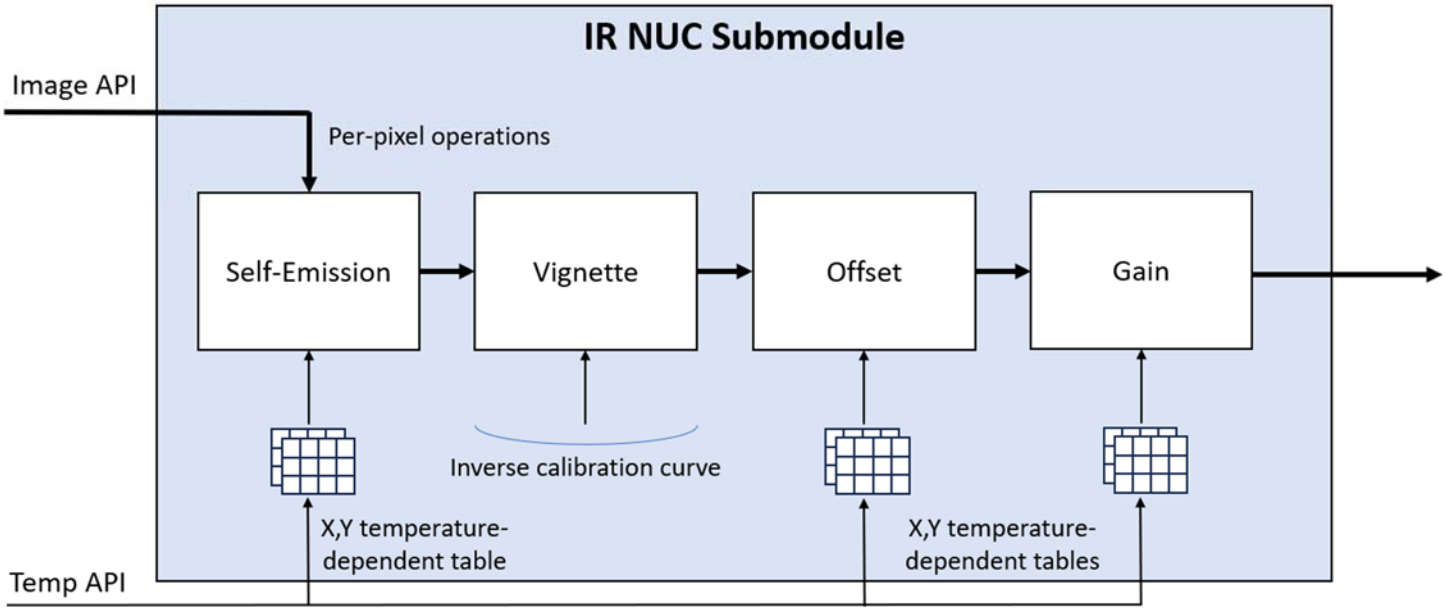
If vehicle pose information is available, it can be used to adjust the center of projection of the virtual camera so that it views the geometry from a different position or orientation due to motion of the vehicle between when the images were captured and the desired rendering time. Also, if egocentric reprojection is being performed so that the viewpoint will be from the pilot's chair rather than from the front of the vehicle, this will be used to adjust the center of projection for the virtual camera. It can also be used to adjust the distortion correction to remove shear and/or stretching of the image due to camera motion during capture.

The image sub-regions are then projected onto the manifold, which is viewed using the virtual camera's center of projection and viewing parameters, producing the requested image to be sent to the Display submodule.

The system can optionally monitor one or more Analysis API connections for event data. The events from each are filtered based on a likelihood threshold and then passed through the distortion correction and geometric registration (and optionally depth) stages to determine their correct 3D pose in the scene.

NUC Submodule

There are three versions of Non-Uniformity Correction (NUC), two for the IR camera and one for the VIS. They each deal with making brightness uniform within cameras (vignette correction, etc.) and between cameras (matching gain and offset) to remove artifacts and provide seamless stitching between cameras. They are then composited onto the imagery, always being displayed in front. The geometry, color, size, and text associated with each is described in the *TR-010_v16_Render_Implementation*.



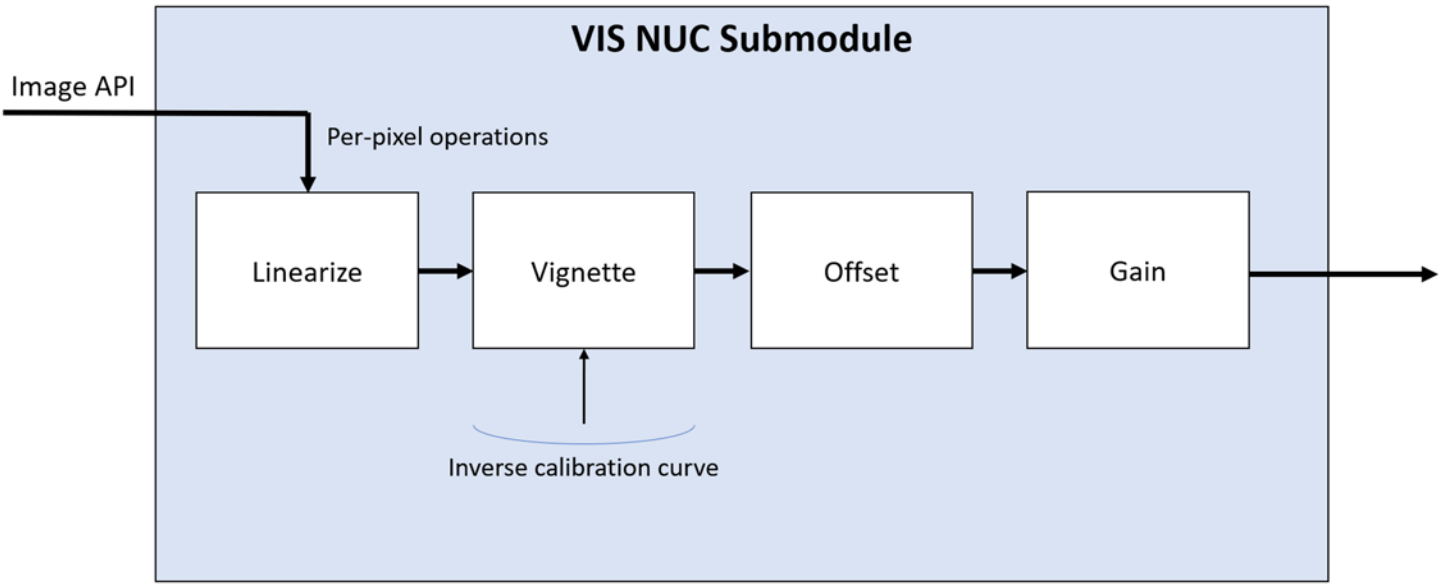
IR analytic NUC: This optional infrared nonuniformity correction submodule compensates for various image-intensity distortions that happen on the camera, producing an image whose values correspond to estimates of the object visible in each pixel that match between cameras. Distortions that can be corrected for (as functions of sensor temperature) include camera self-emission, vignetting, varying per-pixel offsets, varying per-pixel gains, and varying global reference. It does this by including per-camera calibration information (requiring a 2-temperature uniform target across all cameras). For vignetting, this is the inverse of the fractional reduction of intensity at a given radius from the center of the camera. For the others, it consists of a set of tables measured at different sensor temperatures that form a 3D texture along with

the per-pixel X,Y value estimated at that location. Linear interpolation is used in the temperature dimension to interpolate values at temperatures that were not measured.

By mapping all pixels from all cameras to the same brightness space, this submodule enables composition of images from neighboring cameras to match intensity for objects seen by both.

This submodule ***requires input from temperature-controlled per-camera calibration procedures***.

IR scene-based NUC: We will design one or more scene-based NUCs if needed. This will require the addition of an analysis module that watches the incoming video to determine when a new measurement must be triggered.



VIS NUC: This optional visible-light nonuniformity correction submodule performs a subset of the operations of the IR NUC submodule. Rather than temperature dependence, its offset and gain adjustments depend on the auto-gain and auto-exposure values for the camera, which are sent along with each frame. These must be performed in linear space to enable proper mixing of neighboring cameras, so they are all converted into this space by inverting any gamma applied by the camera hardware; this happens in a linearization step that is not present in the IR NUC. The values coming out of the NUC for all cameras in the system are in the same brightness space, having been adjusted differently based on the individual exposure and gain values from the cameras (perhaps only using cameras contributing pixels to the image).

Histograms Submodule

This optional submodule provides a time-stamped intensity histogram for each incoming image from each camera region as they arrive. It is configured at startup to provide a histogram with bins sufficient to store the specified bit depth (8-16) and will truncate the incoming pixels to this many most-significant bits. It computes the normalized histogram, with a floating-point value per bin that holds the fraction of total pixels with this value. It can be configured at run-time to only produce histograms from a subset of the cameras.

It can also be configured at run-time to report one or more summary histograms, configured as above, that store the histograms across a specified subset of cameras.

Because the histograms require all pixels in an image in the general case, they cannot always be computed until the entire image has been received.

Note: For the *mean+std-based* tone-map approach described below, a full histogram is not required. In that case, only the mean and standard deviation of pixel values across all cameras (after they have been adjusted to fall into a common

range) is required. This can be computed much more efficiently than the histogram because of the ability to use shared memory and thread synchronization to replace the use of atomic memory accesses. The module will compute the sum of pixel values and the sum of squared pixel values across each image and then use them to compute the rest of the values, providing them to the *Tone Map* submodule as either per-camera or ensemble mean and standard deviation values.

Depth Submodule

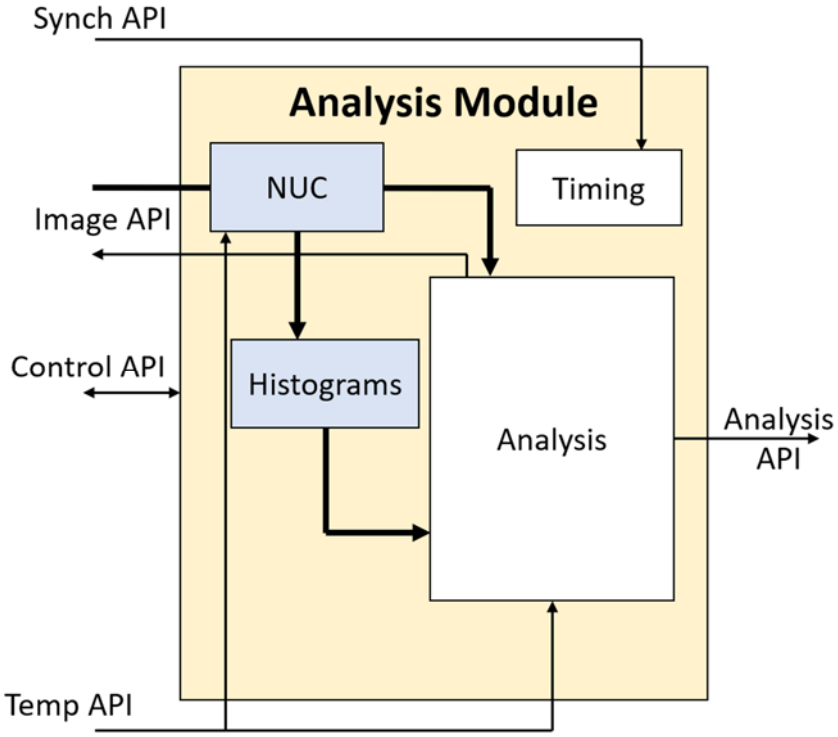
This optional submodule receives full-resolution images from four wide-FOV cameras, two looking to the right and two to the left. Together, these cover the entire field of regard of the narrow-FOV cameras. This module estimates the depth of each pixel using stereo correspondences and perhaps depth from motion, producing a dense 3D manifold whose center of projection is on one of the wide-FOV cameras from each side. This forms a surface to project the high-resolution images from the other cameras in the array onto. This enables stereo display, latency compensation for nearby objects, and egocentric reprojection.

Tone Map Submodule

This optional submodule (the VIS system may not include one) uses the histograms provided by the Histograms submodule and the images provided by the NUC submodule to determine the appropriate mapping from pixel value to display output value for each pixel in each region. Several modes are possible (alone or in combination):

- **Joint-histogram-based:** The histogram for all pixels in all cameras may be used to determine the mapping:
  - **Mean+std-based:** The mean value and standard deviation of the histogram (or one of the means from K-means) may be used to determine the lower and upper bounds to map to the output intensity range, perhaps from X standard deviations below the mean to Y standard deviations above.
  - **AHE:** Adaptive histogram equalization, or **CLAHE** (contrast-limited AHE) may be applied across the entire set of pixels.
- **Gamma correction:** To highlight the lower or higher regions of a selected region, the values (assumed 0-1) may be taken to the power of gamma.
- **Edge based:** In case the NUC does not bring all cameras into the same space, the joint histograms of the pixels at an overlapping edge between two cameras may be used to adjust the offset and/or gain so that they match. The solution for the mesh of neighboring images can be obtained by minimizing the total error across all neighbors.
- **Neural:** A neural tone mapping routine could provide either a global map or a per-pixel mapping across images.

Analysis module



The *Analysis Module* is shown above, along with its submodules and interfaces. (Optional submodules are shown in blue.) Data flow is shown with arrows. (Thicker arrows indicate more data.) An analysis module can also listen to any of the other available Core Module APIs and to additional Analysis APIs as desired.

Most of the APIs used by this module were described in the Core module. The submodules and *Analysis API* are described below.

Timing Submodule

This is the client-side portion of the submodule described in the Core module.

NUC Submodule

This optional submodule is the same as the one described in the Render module.

Histograms Submodule

This optional submodule is the same as the one described in the Render module.

Analysis Submodule

Provides analytics, deep learning, or other analysis on incoming video streams to produce one or more types of analytic reports describing the scene in camera video stream(s). An example is a deep-learning neural network[2] that provides estimates of drone location, type, and relative velocity.

Analysis API

Enables various types of Analysis submodule to send reports to receivers located on the same computer or across a network. Different types of analysis will result in different report types, possibly with multiple report types per type of analysis. This API produces a JSON object per message whose format is described in *Appendix C*. The transport of these objects is described in *TR-020v02_Analysis_API_Implemenation*.

[2] https://en.wikipedia.org/wiki/Deep_learning

Appendix A: Event types and strings

Priority: 1 byte; 0 = system control, 1 = very high, higher are at decreasing levels of importance. Priority 0 and 1 event messages are sent out-of-band, in their own packets.

Type: 3 bytes; 0-255 are reserved for critical errors, types 256-511 are reserved for errors, types 512-767 are reserved for warnings, and higher numbers are informational. The following types are used:

- 256: Invalid operation. String parameters provide integers listing the operation code and its parameters. Sent with very high priority (1).
- 257: Internal error. String parameter with the opcode of the message causing the error. Sent with priority 0.
- 512: Unrecognized opcode. String parameter with the opcode that was unrecognized. Sent with priority 0.
- 768: Clock sync. No string parameters. Always sent with system priority (0). This is sent periodically (10 times per second in version 2.0.0). It can be used by client-side code to determine the offset between the Core Module clock and its own clock.
- 769: Start of replay. String parameter tells the ID of the stored stream that is being replayed. Always sent with system priority (0). This is sent when a stream starts to be replayed. This indicates that the times in the following packets and their messages will be in the time domain of the stored data. Other queued messages are flushed before this message is sent. A clock sync event is always sent immediately following this event, whose value is the time of the first packet that has been stored for replay.
- 770: End of replay. No string parameters. Always sent with system priority (0). This is sent when a stream replay is completed and indicates that the system is going back into live mode. This indicates that times in the following packets and their messages will be in the time domain of the Core Module's current clock. A clock sync event is always sent immediately following this event.
- 771: NUC flag state set: String parameters provide the ID of the camera whose flag has changed and a 0/1 telling the state of the flag (stowed/covering camera). Sent with system priority (0).
- 772: On-camera NUC state: String parameters provide the ID of the camera whose state has changed and a 0/1 telling the state of the on-camera NUC processing (stopped/started). Sent with system priority (0).
- 773: Pause replay. No string parameters. Always sent with system priority (0). This is sent when the replay is paused. Indicates to client code that no additional messages are expected (including state and clock sync messages) until replay resumes or ends.
- 774: Resume replay. No string parameters. Always sent with system priority (0). This is sent when the replay is resumed after being paused. Client code can use this to hard reset its clock synchronization on the next synchronization message (or using the time of the resume message as a clock sync).

Appendix B: Example client-side operation

Example client-side behaviors for different operating conditions follow.

Live flight without recording for simple, unsynchronized system

- Wait for receipt of Discovery message (ID 0)
  - Retrieve IP address and port to send Opcodes to.
  - Verify by looking up the serial # that the system is compatible with rendering.
- Open TCP connection to receive streams to the specified IP and port.
- Send Stream State rate command asking for a report once per second (ID 10) [1.0]
- Wait for receipt of State message (ID 1)
  - Allocate buffers and threads for the number of cameras with appropriate resolutions.
  - Verify that all required submodules are available.
  - Verify that the camera data is streaming, storage and replay are off.
  - Record the number of cameras (NumCams)
- Open UDP port to receive image stream (ImagePort) on the IP that received Discovery (StatusIP)
- Send Stream Subregions Command for each camera (ID 20000) [StatusIP, ImagePort, #, 0, 0, 0, 0, 0, 0, NX-1, NY-1], # is camera number
- Receive and parse Composite frame (20003) messages
  - Sort incoming packets by sequence number to handle UDP re-ordering.
  - Clear back for a given camera each time Begin frame is received for a camera.
  - Copy Frame data to GPU back buffer for each camera.
  - Swap front/back buffers each time a Frame end is received for a camera.
- In a separate thread, render from the front buffer for each camera with the desired viewport.

Add synchronization to minimize latency

- After receipt of Status message:
  - Find the list of internal triggers that any camera is connected to.
  - Find the display update period for the display being used (DispPeriod).
  - Verify that the display period is within the minimum and maximum trigger periods for all cameras.
  - For each internal trigger (ID), send Configure trigger Command (1000) [ID, 3, 1, DispPeriod, Offset, 0.1]
    - Offset configured empirically to allow time for capture, transmission, and client-side operation to occur just before the render loop is released (see below).
  - For each incoming Clock Sync message:
    - Keep track of the minimum offset over the past 100 messages.
    - Fit a line to the minimum from the first set of messages the current set, offset + scale.
- Within the rendering thread
  - Wait until just before vertical retrace (RetraceTime), leaving time to render.
  - Convert RetraceTime to Core Module time since using the offset + scale computed above.
  - Send Software trigger Command (1001) [1, RetraceTimeInCoreModuleTime].
  - Render as before, relying on synchronization to get the data into the front buffers just in time.

Two independent rendering systems could be run on two different Ethernet ports. (At most one of them can perform synchronization if they are to avoid fighting for synchronization.)

Receive temperature information during replay to use as part of NUC

This is very similar to the replay from a live flight. The only difference is as follows:

- After receipt of Status message:
  - Verify that the Temp API is available.
  - Send Stream temperatures Command (ID 40000)
- Watch incoming Temperature messages:
  - Record the current temperature of each camera sensor as it arrives and use it for corrections.

Receive pose information during replay to use for latency correction

This is very similar to the replay from a live flight. The only difference is as follows:

- After receipt of Status message:
  - Verify that the Pose orientation feature is available, and the Pose Position of required.
  - Send Stream poses Command (ID 50000)
- Watch incoming Pose messages:
  - Keep a list of recent Poses along with their times and use the ones that cover the time span from capture to scan-out (along with velocity information) to estimate the adjustments required to offset motion.

Subset cameras during replay to reduce transmission and calculation

This is very similar to the replay from a live flight. The only difference is as follows:

- Within the Composite submodule thread
  - Compare the pilot gaze region with the calibrated camera geometry to determine which rectangular subregions of which cameras it is predicted to overlap at the middle of the frame scan-out time for the next frame after the current frame (SubRegions).
  - Dilate these regions by an amount sufficient to handle distortion correction and prediction error.
  - Send the Stream subregion Command (ID 20000) for each camera that is still visible and Cancel Subregion (ID 20001) for cameras not seen (or just for cameras going out of view).

Read narrow-FOV cameras from one Core and stereo cameras from another

Live: One of the hardware output triggers for the narrow-FOV Core should be sent to both its own cameras and to one of the hardware trigger inputs on the stereo Core. Configure the stereo cameras to trigger with the desired offset from the trigger on the narrow-FOV core.

Connect using two different GigE ports to the two systems and stream each set of cameras to the appropriate Render Submodule(s). Separately perform time alignment with each system.

Replay: Because the clocks on the systems are synchronized, the replay times for each stream will match. The same replay start time is sent to both Cores, causing them to start replay at the same time.

Configure for recording during test flight

The system can be configured to automatically record when powered on. It will record until it runs out of disk space. This can be used to prepare a system to be flown without any attached controls as follows:

- Wait for receipt of Discovery message (ID 0)
  - Retrieve IP address and port to send Opcodes to.
  - Verify by looking up the serial # that the system is compatible with rendering.
- Open TCP stream to send Commands and receive Messages.
- Send Stream State rate (ID 10) [1.0]
- Wait for receipt of Status message (ID 1)
  - Verify that the Storage API is available.
  - Find the list of internal triggers that any camera is connected to.
  - Select a desired capture period (CapPeriod).
  - For each internal trigger (ID), send Configure trigger Command (1000) [ID, 1, 0, CapPeriod, 0.0, 1.0]
- Send Stop replay Command (7) []
- Send Stop recording Command (3) []
- Send Erase all streams Command (30000) []
- Send Start-up recording state Command (8) [1]
- Wait for receipt of two more Status messages (ID 1)
  - Verify that start-up recording state is set to 1.

After this, power down the unit. It can be attached to a vehicle. Power it on a few seconds before the recording should start and power it off after the recording is done.

When it is again powered up on the ground, it will start recording to a new file (if there is available disk space), but this can be turned off by another program that is going to replay it. That program can select the first data set as the one to be replayed.

Replay after recorded flight

This is very similar to the replay from a live flight. The only difference is as follows:

- After receipt of Status message:
  - Verify that the Storage API is available.
  - Send Stream file list Command (ID 30001) [].
  - Wait for Stream List message (30000).
    - Find the first ID in the list (SessionID).
  - Send Stop recording Command (3) [].
  - Send Stop replay Command (7) [].
  - Send Start replay Command (4) [SessionID, Now].

Repeat replay after it completes

To make the system replay the recorded flight after it completes, add the following:

- Watch incoming Status messages:
  - When Replay at end becomes 1:
    - Send Stop replay Command (7) [].
    - Send Start replay Command (4) [SessionID, Now].
- Watch incoming Sync Event messages:
  - When *Start of replay* or *End of replay* messages arrive, reset the clock offset + scale. Initialize the offset using the clock-sync message in the same packet and initialize the scale to 1.0.

Copy data after recorded flight

The *SFTP* protocol is used to remotely access the files on the Storage Module computer. Copy the data files stored for the stream, which will need to be replayed through a Storage Module or separately parsed in an implementation-dependent manner.

Appendix C: Analysis API JSON objects

The Analysis API specifies that JSON objects are emitted based on the analysis of each frame. Each object is unnamed and consists of a dictionary of elements. The elements of the dictionary depend on the type of analysis being performed. The ***CamID***, ***Time***, and ***Name*** elements must be present:

- **CamID**: Camera ID: Integer camera ID that the report is generated for (first camera is 1).
- **Time**: Array of two integers, seconds and microseconds, recording the time of the event in Core Module time. This could be copied from the time field of the image message that described the pixel at the center of the object or they could be based on an object state that is determined from multiple images.
- **Name**: String naming the analysis-dependent name of this object. Each name must be unique across an analysis stream. The same name can be used to track the same object from frame to frame.

The following elements, if they are included, will have these names and formats:

- **Loc**: Image location: List of two floating-point values in the range [0-1] specifying the fraction of the image from the left to right (X) and from top to bottom (Y). The (0,0) location is at center of the pixel at the top left corner of the image and the (1,1) location is at the center of the pixel at the lower-right, pixel [NX-1, NY-1].
- **Rect**: Image extent: List of two positive floating-point values specifying the fraction of the image covered by the object in X and Y, whose center is specified in **Loc**.
- **Vel**: Image velocity: List of two floating-point values specifying the speed of the object in fractions of the image per second in X and Y.
- **Class:** An array of one or more un-named objects. All objects must have a Type field. If there is more than one object in the array, all objects must have a Chance field.
    - **Type**: String naming the analysis-dependent type of the object. Types begin with a unique source identifier followed by underscore. The source identifier is *ASDP_* is reserved for standard types so that multiple sources that produce messages with that type will produce consistent results. The empty type “” represents the null class, the chance that the object is not of any of the other classes.
    - **Chance**: Probability that the event is a member of this class, from 0 to 1. If Chance is present in the classes, the values must sum to less than or equal to 1. If they sum to less than 1, the chance is 1-sum that the object is of the null class (not properly classified).
    - **IFF**: String from the list: “friend”, “foe”, “neutral”, “unknown”.

Additional analysis-dependent elements can be included.